Quatrième partie – Gestion des contraintes avec des triggers :

11) Mise à jour automatique d'un attribut calculé avec un trigger lors d'une insertion.

- On souhaite programmer un trigger afin que l'attribut `nbTotalJourneesTravail` de la table `Salaries` soit mis à jour automatiquement lorsque on insère une nouvelle ligne directement dans la table `Travailler` (pour commencer, on ne gèrera pas les modifications et les suppressions de lignes de cette table).
- Résultat attendu :

```
SELECT nbTotalJourneesTravail
FROM Salaries
WHERE codeSalarie = 'S1';

NBTOTALJOURNEESTRAVAIL
----------------------
6
```

L'ajout d'une journée de travail pour le salarié S1 devrait incrémenter de 1 son nombre total de journées travaillées.

```
INSERT INTO Travailler VALUES ('S1', '10/01/2022', 'P1');

SELECT nbTotalJourneesTravail
FROM Salaries
WHERE codeSalarie = 'S1';

NBTOTALJOURNEESTRAVAIL
----------------------
7
```

12) Gestion des multiplicités maximales des associations avec un trigger.

- On souhaite programmer un trigger qui empêche un salarié d'être affecté à plus de trois équipes lorsqu'on insère une nouvelle ligne directement dans la table `EtreAffecte` (pour commencer on ne gèrera pas les modifications de lignes dans cette table).
- Résultat attendu :

```
SELECT codeSalarie, codeEquipe
FROM EtreAffecte
WHERE codeSalarie IN ('S2', 'S8');

CODESALARIE  CODEEQUIPE
------------ -----------
S2           E2
S2           E3
S8           E2
S8           E3
S8           E4
```

L'instruction suivante doit déclencher **une erreur** car le salarié S8 est déjà affecté à 3 équipes.

```
INSERT INTO EtreAffecte VALUES ('S8', 'E1');
Le salarié est déjà affecté à au moins 3 équipes

SELECT *
FROM EtreAffecte
WHERE codeSalarie = 'S8'
AND codeEquipe = 'E1';

aucune ligne sélectionnée
```

Par contre, l'instruction suivante ne déclenchera pas d'erreur et va rajouter une ligne dans la table `EtreAffecte`.

```
INSERT INTO EtreAffecte VALUES ('S2', 'E4');

SELECT *
FROM EtreAffecte
WHERE codeSalarie = 'S2'
AND codeEquipe = 'E4';

CODESALARIE  CODEEQUIPE
------------ -----------
S2           E4
```

13) Mise à jour des attributs calculés lors des modifications, insertions et suppressions.

- On souhaite modifier le trigger de la question 11, afin que l'attribut `nbTotalJourneesTravail` de la table `Salaries` soit mis à jour automatiquement lorsque on insère une nouvelle ligne dans la table `Travailler`, mais aussi lorsqu'on modifie ou on supprime des lignes de cette table.
- Résultat attendu :

```
SELECT codeSalarie, nbTotalJourneesTravail
FROM Salaries
WHERE codeSalarie IN ('S1', 'S5');

CODESALARIE  NBTOTALJOURNEESTRAVAIL
---------- - ----------------------
S1             7
S5             8
```

La modification du salarié d'une journée de travail doit modifier le nombre total de journées travaillées des deux salariés impactés par la modification.

```
UPDATE Travailler
SET codeSalarie = 'S5'
WHERE codeSalarie = 'S1'
AND dateTravail = '10/01/2022';


SELECT codeSalarie, nbTotalJourneesTravail
FROM Salaries
WHERE codeSalarie IN ('S1', 'S5');

CODESALARIE  NBTOTALJOURNEESTRAVAIL
---------- - ----------------------
S1             6
S5             9
```

La suppression d'une journée de travail du salarié S5 doit décrémenter de 1 son nombre total de journées travaillées.

```
DELETE Travailler
WHERE codeSalarie = 'S5'
AND dateTravail = '10/01/2022';

SELECT nbTotalJourneesTravail
FROM Salaries
WHERE codeSalarie = 'S5';

NBTOTALJOURNEESTRAVAIL
----------------------
8
```

14) Trigger permettant de gérer les contraintes sur un héritage

- On souhaite programmer des triggers qui empêchent un projet d'être à la fois interne et externe lorsqu'on insère une nouvelle ligne directement dans les tables `ProjetsExternes` et `ProjetsInternes`.
- Résultat attendu :

```
SELECT codeProjet, 'EXTERNE' AS typeProjet
FROM ProjetsExternes
UNION
SELECT codeProjet, 'INTERNE' AS typeProjet
FROM ProjetsInternes;

CODEPROJET  TYPEPROJET
----------- -----------
P1          EXTERNE
P2          EXTERNE
P3          EXTERNE
P4          EXTERNE
P5          EXTERNE
P6          INTERNE
P7          INTERNE
P8          INTERNE
```

L ajouter un projet externe avec le code P6 doit déclencher **une erreur** car le projet P6 existe déjà dans la table `ProjetsInternes`

```
INSERT INTO ProjetsExternes (codeProjet, nomProjet, villeProjet,
    clientProjet, codeEquipe) VALUES ('P6', 'Robots de discéditation des
                                      footballeurs sur réseaux sociaux',
                                                'Paris', 'PSG', 'E4');
Insertion impossible, un projet interne possède le même code

SELECT nomProjet
FROM ProjetsExternes
WHERE codeProjet = 'P6';

aucune ligne sélectionnée
```

Par contre, il doit être possible de rajouter un projet P9 dans la table `ProjetsExternes`

```
INSERT INTO ProjetsExternes (codeProjet, nomProjet, villeProjet,
                  clientProjet, codeEquipe) VALUES ('P9', 'Site du BUT',
                                                'Paris', 'IUT', 'E4');

SELECT nomProjet
FROM ProjetsExternes
WHERE codeProjet = 'P9';

NOMPROJET
------------
Site du BUT
```

15) Un autre Trigger !

Les minutes paires, il ne doit pas être possible d'insérer un nouveau salarié dans la table `Salaries` (on ne doit pouvoir insérer un nouveau salarié que les minutes impaires).
Quel type de trigger faut-il utiliser (trigger de ligne ou trigger de table ?). Et pourquoi ?
Ecrire le code de votre trigger et le tester.

16) Trigger et contraintes d'inclusion concernant plus de deux associations.

- On souhaite programmer un trigger qui empêche un salarié de travailler sur un projet qui est réalisé par une équipe dans laquelle n'est pas affecté le salarié en question. On veut que ce trigger se déclenche chaque fois que l'on ajoute une journée de travail d'un salarié (si vous avez du temps, vous pourrez ensuite programmer le trigger pour qu'il se déclenche aussi lorsqu'on modifie une journée de travail déjà existante).
- Résultat attendu :

```
SELECT nbTotalJourneesTravail
FROM Salaries
WHERE codeSalarie = 'S1';

NBTOTALJOURNEESTRAVAIL
----------------------
6
```

Si on indique que le salarié S1 a travaillé le 11/01/2022 sur le projet P1, il ne doit pas y avoir d'erreur.

```
INSERT INTO Travailler VALUES ('S1', '11/01/2022', 'P1');

SELECT nbTotalJourneesTravail
FROM Salaries
WHERE codeSalarie = 'S1';


NBTOTALJOURNEESTRAVAIL
----------------------
7
```

Par contre, si on indique que le salarié S1 a travaillé le 12/01/2022 sur le projet P5, il doit y avoir **une erreur** (car le salarié S1 n'est pas affecté à l'équipe qui travaille sur le projet P5).

```
INSERT INTO Travailler VALUES ('S1', '12/01/2022', 'P5');
Un salarié ne peut pas travailler sur un projet qui est réalisé par
une équipe dans laquelle il n'est pas affecté

SELECT nbTotalJourneesTravail
FROM Salaries
WHERE codeSalarie = 'S1';

NBTOTALJOURNEESTRAVAIL
----------------------
7
```

Cinquième partie – Vues et Triggers :

17) Création d'une vue multitables.

- Créer une vue multitables *Affectations* qui contient toutes les affectations des salariés dans les différentes équipes ; avec pour chaque affectation, le code, le nom et le prénom du salarié ainsi que le code et le nom de l'équipe. Cette vue doit avoir la structure suivante :
  AFFECTATIONS (codeSalarie, nomSalarie, prenomSalarie, codeEquipe, nomEquipe)
- Vérifier que votre vue affiche bien ce qu'il faut. Puis essayer d'insérer des données à travers cette vue. Que se passe-t-il ? Pourquoi ?

```
INSERT INTO Affectations
VALUES ('S9', 'Zétofrais', 'Mélanie', 'E5', 'Indigo');
```

18) Trigger INSTEAD OF

- A l'aide d'un trigger INSTEAD OF, faites qu'il soit possible d'insérer des données à travers la vue multitables précédente. Si le salarié de l'affectation qui est inséré à travers la vue n'existe pas, il doit être créé dans la table *Salariés* (avec un nombre de journées de travail égal à 0). De même, si l'équipe de l'affectation qui est insérée à travers la vue n'existe pas, elle doit être elle aussi créée dans la table *Equipes* (avec un chef d'équipe égal à NULL). Dans tous les cas, une ligne doit être ajoutée dans la table *EtreAffecte*.
- Résultat attendu (les salariés et les équipes indiquées en gras n'existent pas dans la base de données) :

```
INSERT INTO Affectations
VALUES ('S20', 'Zétofrais', 'Mélanie', 'E5', 'Indigo');

INSERT INTO Affectations
VALUES ('S20', 'Zétofrais', 'Mélanie', 'E4', 'Mars');

INSERT INTO Affectations
VALUES ('S5', 'Umule', 'Jacques', 'E20', 'Europa');

INSERT INTO Affectations
VALUES ('S21', 'Zeblouse', 'Agathe', 'E21', 'Galileo');

SELECT * FROM EtreAffecte ORDER BY codeSalarie;

CODESALARIE  CODEEQUIPE
------------ -----------
…            …
S21          E21
…            …
S5           E20
…            …
S20          E4
S20          E5
```

19) Amélioration du Trigger INSTEAD OF

- Modifier votre trigger INSTEAD OF pour qu'une vérification soit effectuée sur la cohérence des données insérées à travers votre vue. Ainsi, si le code du salarié passé en paramètre existe dans la table *Salariés* mais que son nom et son prénom ne correspondent pas à ce qu'il y a dans cette table, rien ne doit être inséré dans la base de données (ni salarié, ni équipe, ni affectation) et une exception est levée. De même, si le code de l'équipe passé en paramètre existe dans la table *Equipes* mais que son nom ne correspond pas à ce qu'il y a dans cette table, rien ne doit être inséré dans la base de données (ni salarié, ni équipe, ni affectation) et une exception est levée.
- Résultat attendu :

```
INSERT INTO Affectations
VALUES ('S20', 'Ouzy', 'Jacques', 'E6', 'Europa');
Les données sur le salarié S20 sont fausses

INSERT INTO Affectations
VALUES ('20', 'Zétofrais', 'Mélanie', 'E20', 'Galileo');
Les données sur l'équipe E20 sont fausses
```

Sixième partie pour les plus rapides – Requêtes SQL faciles :

R1 : le nom et le prénom des salariés qui ont travaillé le 4 janvier 2022 et le 8 janvier 2022 (ils doivent avoir travaillé à ces deux dates).

```
NOMSALARIE  PRENOMSALARIE
----------- --------------
Deuf        John
Umule       Jacques
```

R2 : le nom et le prénom des salariés qui partagent le même prénom qu'un autre salarié.

```
NOMSALARIE  PRENOMSALARIE
----------- --------------
Titouplin   Jean
Menage      Jean
```

R3 : le nombre de salariés affectés à chaque équipe.

```
NOMEQUIPE  NBSALARIES
---------- -----------
Luna       6
Galileo    1
Ganymède   4
Europa     1
Juno       5
Indigo     1
Mars       6
```

R4 : le nom de l'équipe qui possède le plus de salariés affectés.

```
NOMEQUIPE
----------
Luna
Mars
```

R5 : le nom et le prénom des salariés qui sont affectés dans tous les projets externes où le salarié Jean Menage est affecté.

```
NOMSALARIE  PRENOMSALARIE
----------- --------------
Deuf        John
Gator       Ali
Stiké       Sophie
```

R6 : le nom et le prénom des salariés qui sont exactement affectés aux même projets externes que Jean Menage.

```
NOMSALARIE  PRENOMSALARIE
----------- --------------
Gator       Ali
```

R7 : pour chacune des villes où il y a un ou des projets externes réalisés, indiquer le nom du projet qui possède le plus de salariés affectés à son équipe.

```
VILLEPROJET  NOMPROJETJ
------------ ------------------------------------------------
Montpellier  Logiciel Sécurité Sociale
Béziers      Outil CRM de la Poste
Nimes        Site du club de billes des crocodiles ramollis
Paris        Site du BUT
```

R8 : le nom et le prénom des salariés qui sont des Ninjas SQL.

```
NOMSALARIE  PRENOMSALARIE
----------- --------------------------
Palleja     Xavier
Palleja     Nathalie
```